

Mini Sebenta

Robótica Médica — Braço Planar de 2 Elos

Contexto geral

O robô é um manipulador planar de 2 elos, usado para cirurgia ocular. Tem duas juntas rotativas (ombro q_1 e cotovelo q_2), e a ponta do braço (end-effector) interage com o olho.

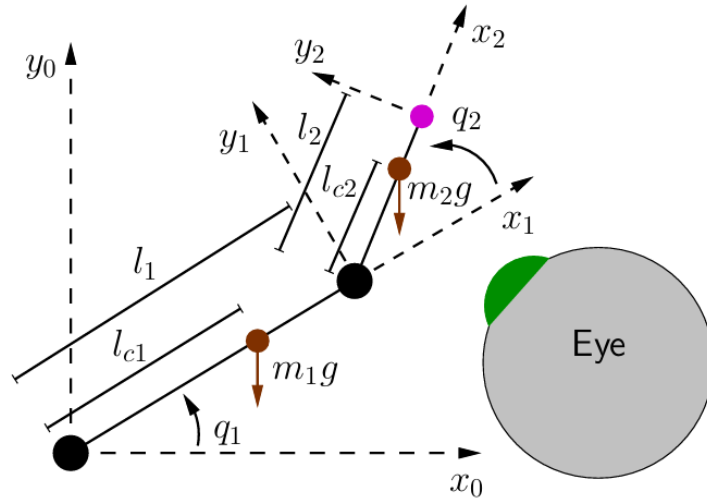


Figura 1 — Robô planar de 2 elos para cirurgia ocular.

Para descrever o movimento deste robô, respondem-se 3 perguntas, cada uma mais complexa que a anterior:

- Cinemática direta (ex 1.1) — Onde está a ponta do braço? (ângulos $\rightarrow$ posição x, y)
- Jacobiano (ex 1.2) — A que velocidade se move? (velocidade das juntas $\rightarrow$ velocidade linear/angular)
- Dinâmica (ex 1.3) — Que torque é preciso aplicar? (torque dos motores $\rightarrow$ aceleração das juntas)

1. Cinemática Direta (ex 1.1)

Teoria

O elo 1 (comprimento l_1) faz ângulo q_1 com a horizontal. O elo 2 (comprimento l_2) não faz ângulo q_2 com a horizontal — faz ângulo q_2 em relação ao elo 1. Por isso, em relação à base, o elo 2 aponta na direção q_1+q_2 (os ângulos somam-se).

$$x = l_1 \cdot \cos(q_1) + l_2 \cdot \cos(q_1+q_2) \quad y = l_1 \cdot \sin(q_1) + l_2 \cdot \sin(q_1+q_2)$$

Código

```
%% Forward kinematics, ex 1.1
function [x,y]= forward_kinematics(q1,q2,l1,l2)
    x = l1*cos(q1) + l2*cos(q1+q2);
    y = l1*sin(q1) + l2*sin(q1+q2);
end
```

Para que serve: é a base de tudo — sem saber onde está a ponta, não se consegue calcular velocidades nem dinâmica.

2. Jacobianos (ex 1.2)

Teoria

Se $q_1(t)$ e $q_2(t)$ mudam no tempo, x e y também mudam. Pela regra da cadeia:

$$\dot{x} = (\partial x / \partial q_1) \cdot \dot{q}_1 + (\partial x / \partial q_2) \cdot \dot{q}_2$$

Escrito em forma matricial: $[\dot{x}; \dot{y}] = J(q) \cdot [\dot{q}_1; \dot{q}_2]$. J é uma "tabela de sensibilidades" — não é constante, muda com a postura do braço.

Porque há 5 Jacobianos diferentes

- End-effector (J_e) — controla a ponta do braço
- Centros de massa ($J_{v,c1}$, $J_{v,c2}$) — necessários para a energia cinética de translação de cada elo (ex 1.3)
- Angulares ($J_{w,1}$, $J_{w,2}$) — necessários para a energia cinética de rotação de cada elo

Linear vs. Angular

Linear (J_v): velocidade de translação do ponto (depende de $\sin$, $\cos$). Angular (J_w): velocidade de rotação do elo em torno do eixo z (constante, porque o robô é planar — nunca sai do plano xy , logo só pode rodar em torno do eixo perpendicular a esse plano).

$$w_1 = \dot{q}_1 \quad w_2 = \dot{q}_1 + \dot{q}_2$$

(o elo 2 é "arrastado" pela rotação da junta 1, e roda ainda mais com a junta 2)

Código

```
%% Jacobian end-effector, ex 1.2
function J = jacobian_ee(q1, q2, l1, l2)
```

```

    J = [-l1*sin(q1) - l2*sin(q1+q2),    -l2*sin(q1+q2);
          l1*cos(q1) + l2*cos(q1+q2),     l2*cos(q1+q2)];
end

```

```

%% Jacobian linear, centro de massa 1, ex 1.2
% xc1 = lc1*cos(q1), yc1 = lc1*sin(q1) -> só depende de q1
function J = jacobian_cm1(q1, lc1)
    J = [-lc1*sin(q1),    0;
          lc1*cos(q1),    0];
end

```

```

%% Jacobian linear, centro de massa 2, ex 1.2
function J = jacobian_cm2(q1, q2, l1, lc2)
    J = [-l1*sin(q1) - lc2*sin(q1+q2),    -lc2*sin(q1+q2);
          l1*cos(q1) + lc2*cos(q1+q2),     lc2*cos(q1+q2)];
end

```

```

%% Jacobian angular, elo 1, ex 1.2
function Jw = jacobian_cm1_w()
    Jw = [1, 0];
end

```

```

%% Jacobian angular, elo 2, ex 1.2
function Jw = jacobian_cm2_w()
    Jw = [1, 1];
end

```

Para que serve: são os "blocos de construção" da matriz de inércia $M(q)$ no exercício 1.3.

3. Dinâmica — $M(q)$, $C(q,\dot{q})$, $g(q)$ (ex 1.3)

A equação central do projeto

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau$$

(inércia) (efeitos cruzados) (peso)

3.1 $M(q)$ — a inércia

Intuição: tal como um carrinho de compras cheio resiste mais a acelerar do que um vazio, o braço robótico tem uma "resistência a acelerar" que muda consoante a postura (braço esticado vs. dobrado sente inércias diferentes).

Como se calcula: soma-se a energia cinética de translação (via J_v) e rotação (via J_w) de cada elo:

$$K = \frac{1}{2} \dot{q}^T M(q) \dot{q}, \quad M(q) = \sum_i [m_i J_{v,ci}^T J_{v,ci} + I_i J_{\omega,i}^T J_{\omega,i}]$$

```
%% Mass matrix M(q), ex 1.3
% Reaproveita as Jacobianas do ex 1.2 (projeto iterativo)
function M = massmatrix(q1, q2, m1, m2, l1, lc1, lc2, I1, I2)
    Jvc1 = jacobian_cm1(q1, lc1);
    Jvc2 = jacobian_cm2(q1, q2, l1, lc2);
    Jw1 = jacobian_cm1_w();
    Jw2 = jacobian_cm2_w();

    M = m1*(Jvc1.')*Jvc1 + I1*(Jw1.')*Jw1 + ...
        m2*(Jvc2.')*Jvc2 + I2*(Jw2.')*Jw2;
end
```

Resultado validado ($q_1=\pi/3$, $q_2=5\pi/18$, valores do enunciado):

$$M = [0.0232 \quad 0.0033 ; 0.0033 \quad 0.0013]$$

Simétrica ✓, $M_{11} > M_{22}$ ✓ (a junta do ombro move o braço todo, sente mais inércia que o cotovelo).

3.2 $C(q,\dot{q})$ — os efeitos cruzados (Coriolis/centrípetos)

Intuição: como numa cadeira giratória — se encolheres os braços enquanto rodas, aceleras sozinho, sem ninguém te empurrar. Isto acontece porque a tua "distribuição de massa" (equivalente a $M(q)$) mudou enquanto te mexias. O mesmo acontece no robô: como $M(q)$ não é constante, essa mudança gera forças extra nas juntas.

Como se calcula (símbolos de Christoffel): deriva-se $M(q)$ sistematicamente — não há física nova, só diferenciação:

$$C_{kj} = \sum_i \frac{1}{2} (\frac{\partial M_{kj}}{\partial q_i} + \frac{\partial M_{ki}}{\partial q_j} - \frac{\partial M_{ij}}{\partial q_k}) \dot{q}_i$$

```
%% Coriolis matrix C(q,qdot), ex 1.3
function C = coriolis_matrix(q1, q2, q1dot, q2dot, m1, m2, l1, lc1, lc2, I1, I2)
    M = massmatrix(q1, q2, m1, m2, l1, lc1, lc2, I1, I2);
    q = [q1, q2];
    qdot = [q1dot, q2dot];
    n = 2;
    C = sym(zeros(n,n));
    for k = 1:n
        for j = 1:n
            for i = 1:n
                c_ijk = 0.5*( diff(M(k,j), q(i)) + diff(M(k,i), q(j)) -
diff(M(i,j), q(k)) );
                C(k,j) = C(k,j) + c_ijk*qdot(i);
            end
        end
    end
```

```

        end
    end
end
C = simplify(C);
end

```

Resultado validado:

$$C = [-l1 \cdot lc2 \cdot m2 \cdot \dot{q}2 \cdot \sin(q2) \quad -l1 \cdot lc2 \cdot m2 \cdot \sin(q2) \cdot (\dot{q}1 + \dot{q}2) ; \quad l1 \cdot lc2 \cdot m2 \cdot \dot{q}1 \cdot \sin(q2) \quad 0]$$

Todos os termos têm $\sin(q2)$ ✓ (vêm da derivada do $\cos(q2)$ em M); $C22=0$ ✓ (a junta 2 não gera Coriolis sobre si mesma).

3.3 g(q) — o peso

Intuição: é só $E_p = m \cdot g \cdot h$ aplicado a cada elo — quanto mais alto está o centro de massa, mais energia potencial, mais "puxão" da gravidade sentido nas juntas.

$$U(q) = \sum_i m_i \cdot g \cdot y_{ci}, \quad g(q) = \partial U / \partial q$$

```

%% Gravity term g(q), ex 1.3
function g_vec = gravity(q1, q2, m1, m2, l1, lc1, lc2, g_acc)
    yc1 = lc1*sin(q1);
    yc2 = l1*sin(q1) + lc2*sin(q1+q2);
    U = m1*g_acc*yc1 + m2*g_acc*yc2;
    g_vec = jacobian(U, [q1, q2]).';
    g_vec = simplify(g_vec);
end

```

Resultado validado:

$$g(q) = [m2 \cdot g \cdot (lc2 \cdot \cos(q1+q2) + l1 \cdot \cos(q1)) + m1 \cdot g \cdot lc1 \cdot \cos(q1) ; \quad m2 \cdot g \cdot lc2 \cdot \cos(q1+q2)]$$

A segunda linha só tem $m2$ ✓ (a junta 2 só sente o peso do elo 2).

3.4 Checks de consistência (confirmam que M e C estão corretos)

- $M - M^T = 0 \rightarrow M$ é simétrica (propriedade obrigatória) — ✓ confirmado
- $\dot{M} - 2C$ é antissimétrica $\rightarrow C$ não injeta/retira energia artificialmente — ✓ confirmado

```

Mdot = diff(M,q1)*q1dot + diff(M,q2)*q2dot;
N = Mdot - 2*C;
simplify(N + N.') % deve dar [0 0; 0 0]

```

3.5 Versão completa (3D) — comparação com a abordagem do professor

O professor mostrou na aula uma versão mais geral de $M(q)$, usando Jacobianos 3×2 (com uma linha extra para a componente z, sempre nula no caso planar), o tensor de inércia completo (3×3) e matrizes de rotação $R1, R2$ para rodar a inércia do referencial do elo para o referencial da base:

$$M(q) = \sum_i [m_i \cdot J_{v,ci}^T \cdot J_{v,ci} + J_{\omega,i}^T \cdot (R_i \cdot I_{ic} \cdot R_i^T) \cdot J_{\omega,i}]$$

Foi implementada uma segunda função, `massmatrix_full.m`, que reaproveita as mesmas Jacobianas do exercício 1.2 (sem as reescrever), acrescentando apenas a linha de zeros (componente z) por fora e o tensor de inércia/rotação completos:

```

%% Mass matrix M(q), versão completa (3D), ex 1.3

```

```

function M = massmatrix_full(q1, q2, m1, m2, l1, lc1, lc2, I1czz, I2czz)
    % Reaproveita as Jacobianas do ex 1.2, só acrescenta a linha de z
    Jvc1_2d = jacobian_cm1(q1, lc1);
    Jvc2_2d = jacobian_cm2(q1, q2, l1, lc2);
    Jvc1 = [Jvc1_2d; 0, 0];
    Jvc2 = [Jvc2_2d; 0, 0];

    Jw1_1d = jacobian_cm1_w();
    Jw2_1d = jacobian_cm2_w();
    Jw1 = [0, 0; 0, 0; Jw1_1d];
    Jw2 = [0, 0; 0, 0; Jw2_1d];

    % Tensores de inércia (3x3) -> só a componente zz é não-nula
    I1c = [0 0 0; 0 0 0; 0 0 I1czz];
    I2c = [0 0 0; 0 0 0; 0 0 I2czz];

    % Matrizes de rotação (em torno de z)
    R1 = [cos(q1) -sin(q1) 0; sin(q1) cos(q1) 0; 0 0 1];
    R2 = [cos(q1+q2) -sin(q1+q2) 0; sin(q1+q2) cos(q1+q2) 0; 0 0 1];

    M = m1*(Jvc1.')*Jvc1 + (Jw1.')*R1*I1c*(R1.')*Jw1 + ...
        m2*(Jvc2.')*Jvc2 + (Jw2.')*R2*I2c*(R2.')*Jw2;
    M = simplify(M);
end

```

Teste de consistência — comparando as duas versões simbolicamente:

```

syms q1 q2 m1 m2 l1 lc1 lc2 I1 I2 real
M_simples = massmatrix(q1, q2, m1, m2, l1, lc1, lc2, I1, I2);
M_completa = massmatrix_full(q1, q2, m1, m2, l1, lc1, lc2, I1, I2);
diff_M = simplify(M_simples - M_completa)

% Resultado obtido:
% diff_M =
% [0, 0]
% [0, 0]

```

A diferença deu identicamente zero — confirma-se que, no caso planar, $R_i \cdot I_{ic} \cdot R_i^T = I_{ic}$ (rodar a inércia em torno do seu próprio eixo de simetria não a altera), pelo que a forma geral se reduz exatamente ao uso direto do escalar I_i . As duas versões ficam no projeto: `massmatrix.m` (simples, usada no resto do trabalho, mais rápida) e `massmatrix_full.m` (completa/formal, serve de justificação e prova da simplificação).

4. Mapa mental — como tudo se liga

```
Cinemática direta (ex 1.1)
  |
  v
Jacobianos: lineares + angulares (ex 1.2)
  |
  v
M(q) <- soma de mi*Jvci'*Jvci + Ii*Jwi'*Jwi (energia cinética)
  |
  v
C(q,qdot) <- derivadas de M (simbolos de Christoffel)
  |
  v
g(q) <- derivada de U (energia potencial, altura dos c.m.)
  |
  v
M(q)qdd + C(q,qdot)qdot + g(q) = tau (equacao de movimento)
  |
  v
Simulink: simular o robo (ex 1.4)
```

Ideia-chave do projeto: cada exercício é um degrau que alimenta o seguinte — nada se recalcula do zero, tudo se reaproveita (código iterativo).

5. Glossário rápido

Termo	Significado
Elo (link)	a barra física rígida do braço
Junta (joint)	o ponto de rotação, onde um motor atua
End-effector	a ponta livre do braço (onde se agarra/opera)
Centro de massa	o ponto onde está "concentrado" o peso de um elo
q_1, q_2	ângulos das juntas (posição)
$\dot{q}_1, \dot{q}_2$	velocidades angulares das juntas
J (Jacobiano)	relaciona velocidade das juntas com velocidade de um ponto
M(q)	matriz de inércia — "preguiça" do robô, depende da postura
C(q, $\dot{q}$)	efeitos de Coriolis/centrípetos — inércia a mudar em tempo real
g(q)	torque de gravidade sentido em cada junta
τ	torque aplicado pelos motores

6. Parâmetros do enunciado (robô de 2 elos)

$$m1=0.3 \text{ kg}, m2=0.1 \text{ kg}, l1=0.3 \text{ m}, l2=0.2 \text{ m}$$

$$lc1=l1/2=0.15 \text{ m}, lc2=l2/2=0.1 \text{ m}, Fv=0.04$$

$$I_i = (1/12) \cdot m_i \cdot l_i^2 \rightarrow I1 \approx 0.00225, I2 \approx 0.000333$$

7. Próximos passos do projeto

Estado	Exercício
✓	Ex 1.1 — Cinemática direta
✓	Ex 1.2 — Jacobianos (end-effector + 2 centros de massa, linear + angular)
✓	Ex 1.3 — $M(q)$, $C(q, \dot{q})$, $g(q)$ via Lagrange (falta só a comparação com a versão de IA)
□	Ex 1.4 — Simulador em Simulink ($q0=[\pi/2, \pi/4]^T$, $\dot{q}0=[0,0]^T$, testar $\tau_c=\tau_e=0$)
□	Ex 1.5 — Aplicar $\tau=g(q)$ (compensação de gravidade)
□	Ex 1.6 a 1.11 — Controle de forças, posição, compliance, impedância, admitância
□	Secção 3 — Trajetória para robô de 3 elos (cirurgia abdominal)
□	Secção 4 — Investigação livre (robô Ufactory Lite 6)